Longest Common Subsequence (LCS) Visualizer

1. Introduction

The Longest Common Subsequence (LCS) problem is a classic computer science problem that finds applications in various domains including bioinformatics (DNA sequence alignment), version control systems (diff utility), and natural language processing.
A subsequence is a sequence of characters that appears in the same relative order but not necessarily contiguously. For example, "ACE" is a subsequence of "ABCDE", but "AEC" is not. A common subsequence of two strings is a subsequence that appears in both strings. The Longest Common Subsequence is the longest such common subsequence.

2. Mathematical Definition

Given two sequences $X = x_1, x_2, ..., x_m$ and $Y = y_1, y_2, ..., y_n$, find the longest sequence $Z = z_1, z_2, ..., z_k$ such that Z is a subsequence of both X and Y.

3. Dynamic Programming Approach

3.1 Recurrence Relation

The key insight behind the DP solution is optimal substructure: the LCS of two sequences contains the LCS of their prefixes. We define L[i][j] as the length of the LCS of the prefixes X[1..i] and Y[1..j].
Base Case: If either sequence is empty, the LCS length is 0:
L[i][j] = 0 if i = 0 or j = 0
Recursive Case — Characters are the same: If the last characters of both prefixes match ($x_i = y_j$), then this character must be part of the LCS. We extend the LCS of the shorter prefixes by 1:
L[i][j] = L[i-1][j-1] + 1 if $x_i = y_j$
Recursive Case — Characters are different: If the last characters do not match ($x_i \neq y_j$), then at least one of x_i or y_j is not part of the LCS. We take the maximum of the cell above and the cell to the left, which represents the best LCS we can

achieve by dropping one character from either sequence:
L[i][j] = max(L[i-1][j], L[i][j-1]) if $x_i \neq y_j$

3.2 Understanding the DP Table

The DP table is filled row by row, left to right, using the following rule:

1. Initialize the first row and first column to 0 (base cases — comparing against an empty sequence).
2. For each cell (i, j), compare characters X[i] and Y[j]:
   - If the characters are the same, the value is the diagonal cell (i-1, j-1) plus 1.
   - If the characters are different, the value is the maximum of the cell above (i-1, j) and the cell to the left (i, j-1).
3. The final answer (the LCS length) is obtained in the bottom-right cell L[m][n].

Example: For X = "AB" and Y = "AC":

	Ø	A
Ø	0	0
A	0	1
B	0	1

- Cell (1,1): A = A → match, so L[1][1] = L[0][0] + 1 = 1
- Cell (1,2): A ≠ C → no match, so L[1][2] = max(L[0][2], L[1][1]) = max(0, 1) = 1
- Cell (2,1): B ≠ A → no match, so L[2][1] = max(L[1][1], L[2][0]) = max(1, 0) = 1
- Cell (2,2): B ≠ C → no match, so L[2][2] = max(L[1][2], L[2][1]) = max(1, 1) = 1

The LCS is "A" with length 1.

3.3 Time and Space Complexity

- Time Complexity: O(mn) where m and n are lengths of sequences
- Space Complexity: O(mn) for storing the DP table

4. Optimizations

4.1 Rolling Array Optimization

Instead of storing the entire m × n table, we can store only two rows at a time:

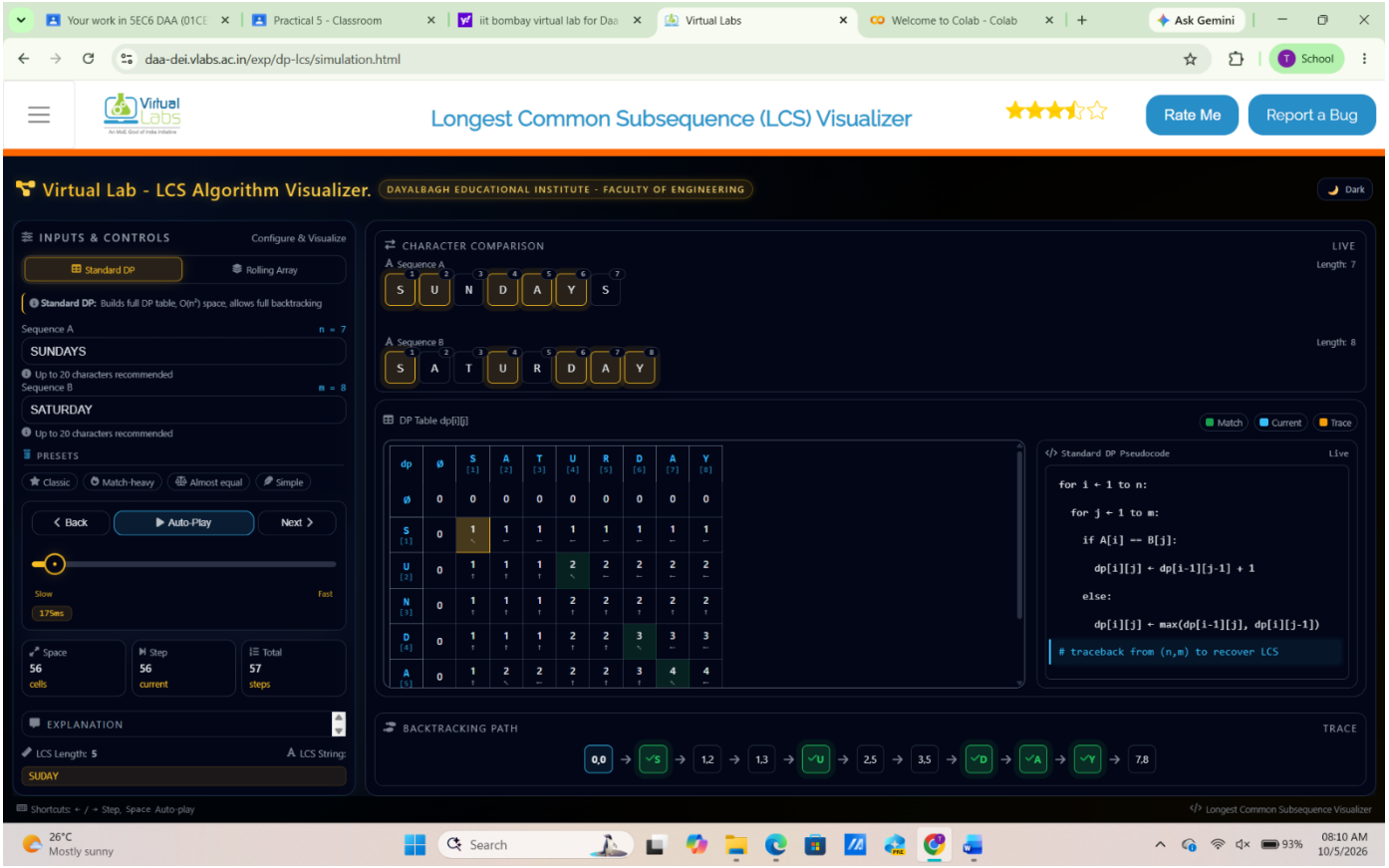
- Space complexity reduces to O(min(m, n))
- Still maintains O(mn) time complexity

5. Applications

1. Bioinformatics: DNA and protein sequence alignment
2. Version Control: Comparing files in git/svn
3. Plagiarism Detection: Document similarity analysis
4. Speech Recognition: Word sequence alignment

Standard DP:

Simulation



Rolling Array:

Simulation

